



Faculté des Sciences El Jadida

Université Chouaib Doukali

Département d'Informatique

Master Business intelligence & Big Data Analytics

Rapport de Projet

Conversion d'Exigences en Logiciels Fonctionnels en Java avec LLM et Agents Intelligents

Module : Agent Intelligent et Systèmes Multi-Agents



Encadré par : Driss Abada

Réalisé par :

El Aouni Hajar

Ifadaen Abed-el Hak

Année Universitaire : 2024-2025

Résumé

Ce projet vise à convertir des exigences en logiciels fonctionnels en utilisant des technologies avancées comme Crew AI et le LLM Gemini Flash. En exploitant la puissance des modèles de langage large et des agents intelligents, nous avons développé un système automatisé et collaboratif pour le développement logiciel.

Les agents créés dans ce projet incluent :

- **Agent d'analyse des exigences** : Responsable de l'identification et de la classification des besoins fonctionnels et non fonctionnels.
- **Agent de planification et décomposition des tâches** : Facilite la structuration des exigences en tâches réalisables.
- **Agent des générateurs de code** : Génère du code source basé sur les spécifications.
- **Agent de validation de test** : Vérifie la conformité du code avec les exigences en exécutant des tests automatisés.
- **Agent de documentation** : Génère automatiquement des documents clairs et détaillés pour les livrables.

Le projet améliore l'efficacité du cycle de vie du développement logiciel grâce à l'automatisation, tout en réduisant les erreurs humaines. Les résultats attendus comprennent une livraison rapide et fiable de logiciels alignés sur les besoins du client.

Table des matières

Introduction	4
1 Etat de l'art	5
1.1 Approche des LLM et des agents intelligents	5
1.2 Projets existants dans le domaine	5
2 Concepts Théoriques et Fondements du Projet	7
2.1 Modèles de langage large (LLM)	7
2.2 Agents intelligents	8
2.3 Java comme langage de programmation	9
2.4 Crew AI	10
3 Conception et Rôles des Agents Intelligents	12
3.1 Requirement Analysis Agent (Agent d'Analyse des Exigences)	13
3.2 Task Planner Agent (Agent de Planification des Tâches)	13
3.3 Code Generator Agent (Agent Générateur de Code)	13
3.4 Test Validator Agent (Agent de Validation des Tests)	13
3.5 Documentation Agent (Agent de Documentation)	13
3.6 Coordination et Intégration des Outils	14
3.7 Résumé du Flux Global	14
4 technologies utilisee	15
4.1 Choix du Modèle de Langage	15
4.2 API de recherche web	15
4.3 Python	16
4.4 Réalisation d'interface en utilisant Flask	17
5 Implémentation	18
5.1 Configuration des clés API	18
5.1.1 Création et Configuration de la clé API Google Gemini	18
5.1.2 Création et Configuration de la Clé API Serper	18
5.2 Implémentation des Agents	19
5.3 Processus Séquentiel dans <code>app.py</code>	21
5.4 Traitement Intermédiaire : Création des Tâches et des Outils	24
5.4.1 Création des Tâches (<code>task.py</code>)	24
5.4.2 Création des Outils (<code>tools.py</code>)	24
5.4.3 Importance de ces Étapes Intermédiaires	25
5.5 Développement de l'interface utilisateur	26

6	Résultats de l'Application	27
6.1	Tests de l'application	27
6.1.1	Exemple 1 : application de calculatrice	28
6.1.2	Exemple 2 : Système de gestion des tâches	33
6.2	Discussion des résultats et améliorations futures	37
6.2.1	Discussion des résultats	37
6.2.2	Améliorations possibles et mises à jour futures	38
7	Conclusion générale	40
8	Références	41

Introduction

La transformation des exigences fonctionnelles en code exécutable est une phase cruciale dans le développement d'une application logicielle, mais elle représente également une tâche complexe et sujette à des erreurs humaines. Les exigences, souvent rédigées en langage naturel par les utilisateurs finaux ou les clients, doivent être converties en spécifications techniques avant de pouvoir être traduites en code source. Ce processus, bien qu'indispensable, peut entraîner un écart entre ce qu'un client demande et ce qui est effectivement livré, engendrant ainsi des délais et des coûts supplémentaires dans le cycle de développement.

Le projet que nous présentons vise à résoudre ces problématiques en exploitant des technologies avancées telles que les modèles de langage large (LLM) et les systèmes multi-agents pour automatiser la conversion des exigences en logiciels fonctionnels. Grâce à l'utilisation de technologies LLM, comme Gemini Flash, et à l'intelligence collective des agents autonomes, nous avons conçu un système permettant de générer automatiquement du code source en Java à partir de spécifications fonctionnelles, réduisant ainsi la probabilité d'erreurs humaines et améliorant l'efficacité du processus de développement.

Ce système automatisé et collaboratif repose sur plusieurs agents, chacun ayant une fonction spécifique dans la gestion du cycle de vie de l'application. L'agent d'analyse des exigences, par exemple, est chargé d'extraire et de structurer les besoins fonctionnels et non fonctionnels, tandis que d'autres agents se consacrent à la planification, à la génération de code, à la validation des tests et à la production de la documentation technique.

Chapitre 1

Etat de l'art

La génération automatique de code vise à automatiser la création de code source à partir de spécifications ou d'exemples. Les approches actuelles incluent l'utilisation de modèles de langage, de techniques d'apprentissage automatique et d'agents intelligents. Bien que ces méthodes offrent des avantages en termes de rapidité et de réduction des erreurs humaines, elles présentent également des limitations. Par exemple, la difficulté à comprendre le contexte complet des exigences peut entraîner des erreurs ou des incohérences dans le code généré. De plus, la complexité des systèmes logiciels modernes nécessite une compréhension approfondie des interactions entre les différents composants, ce qui reste un défi pour les modèles actuels.

1.1 Approche des LLM et des agents intelligents

L'intégration des LLM et des agents intelligents dans le processus de génération de code offre des solutions potentielles à ces défis. Les LLM peuvent interpréter des exigences en langage naturel et générer des fragments de code correspondants, tandis que les agents intelligents peuvent orchestrer le processus en analysant les exigences, en planifiant les étapes de développement, en générant le code, en effectuant des tests et en apportant des corrections si nécessaire. Cette approche intégrée permet de surmonter certaines des limitations des méthodes traditionnelles en offrant une compréhension contextuelle améliorée et une automatisation accrue du processus de développement logiciel.

1.2 Projets existants dans le domaine

Plusieurs projets ont exploré l'utilisation des LLM et des agents intelligents pour la génération automatique de code. Par exemple, GitHub Copilot, basé sur OpenAI Codex, assiste les développeurs en suggérant des extraits de code et en automatisant certaines tâches de programmation. Cependant, malgré leurs avancées, ces modèles rencontrent des défis, notamment en matière de compréhension approfondie des exigences spécifiques d'un projet et de génération de code sans erreurs. Des initiatives récentes, comme le lancement de CriticGPT par OpenAI, visent à améliorer la précision du code généré par les LLM en identifiant et en corrigeant les erreurs potentielles.

Dans le domaine des agents intelligents, CrewAI se distingue en permettant la création d'agents capables de comprendre et de générer du langage naturel, facilitant ainsi la génération de code à partir d'exigences en langage naturel. Cette approche offre une interface sans code, rendant l'automatisation accessible à un public plus large.

En combinant les capacités des LLM et des agents intelligents, ces projets visent à transformer le processus de développement logiciel en le rendant plus rapide, plus précis

et moins dépendant de l'intervention humaine directe.

Chapitre 2

Concepts Théoriques et Fondements du Projet

2.1 Modèles de langage large (LLM)

Les modèles de langage large (LLM) sont des algorithmes d'apprentissage profond conçus pour comprendre et générer du langage naturel. Parmi les LLM notables, on trouve :

- **GPT (Generative Pre-trained Transformer)** : Développé par OpenAI, GPT est capable de générer du texte cohérent et contextuellement pertinent, facilitant des tâches telles que la rédaction automatique et la réponse aux questions.



FIGURE 2.1 – Image de logo Gpt

- **LLaMA (Large Language Model Meta AI)** : Développé par Meta, LLaMA est une famille de modèles de langage conçus pour être efficaces et performants, même avec des ressources computationnelles limitées.



FIGURE 2.2 – Image LLaMA Logo

- **Gemini** : Développé par DeepMind, Gemini est une famille de modèles de langage multimodaux avancés, conçus pour des interactions rapides et contextuelles, intégrant des capacités de compréhension et de génération de texte et d'autres formes de données.



FIGURE 2.3 – Image de Logo Gemini

Avantages pour la conversion d'exigences

- **Compréhension contextuelle** : Les LLM peuvent analyser des descriptions textuelles complexes pour extraire des besoins fonctionnels et non fonctionnels.
- **Génération automatisée** : Ils permettent de traduire les exigences en code ou en documentation, accélérant ainsi le processus de développement.
- **Adaptabilité** : Les LLM peuvent être ajustés pour répondre aux spécificités de divers domaines d'application, assurant une pertinence accrue des solutions générées.

2.2 Agents intelligents

Les agents intelligents sont des systèmes autonomes capables de percevoir leur environnement, de raisonner et de prendre des décisions pour atteindre des objectifs spécifiques.

Fonctionnement des agents intelligents

- **Perception** : Les agents collectent des informations sur leur environnement via des capteurs ou des données d'entrée.
- **Décision** : Ils traitent ces informations pour déterminer la meilleure action à entreprendre en fonction de leurs objectifs.
- **Action** : Les agents exécutent des actions qui influencent l'environnement ou interagissent avec d'autres agents ou utilisateurs.

Mélange entre agents et LLM

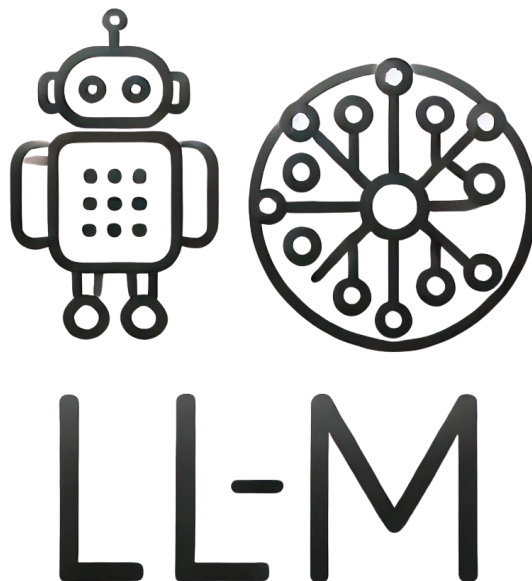


FIGURE 2.4 –

- **Collaboration** : Les LLM fournissent des analyses contextuelles approfondies, tandis que les agents intelligents utilisent ces informations pour prendre des décisions éclairées et exécuter des tâches spécifiques.
- **Automatisation avancée** : La combinaison des LLM et des agents intelligents permet d'automatiser des processus complexes, améliorant ainsi l'efficacité opérationnelle.
- **Amélioration continue** : Les agents peuvent apprendre des interactions précédentes, surtout lorsqu'ils sont intégrés avec des LLM, pour affiner leurs performances au fil du temps.

2.3 Java comme langage de programmation

Java est un langage de programmation orienté objet, reconnu pour sa portabilité, sa robustesse et sa vaste communauté de développeurs.

Avantages pour le développement logiciel

- **Portabilité** : Grâce à la machine virtuelle Java (JVM), les applications Java peuvent s'exécuter sur diverses plateformes sans modification du code source.



FIGURE 2.5 – Image de logo Java

- **Écosystème riche** : Java dispose d'une multitude de bibliothèques et de frameworks facilitant le développement rapide et efficace d'applications complexes.
- **Sécurité et fiabilité** : Avec sa gestion automatique de la mémoire et ses fonctionnalités de sécurité intégrées, Java est idéal pour le développement d'applications robustes.

Intégration avec les technologies de LLM et agents intelligents

- **Interopérabilité** : Java peut intégrer des modèles de langage via des API, permettant l'utilisation de LLM dans des applications Java.
- **Frameworks dédiés** : Des bibliothèques comme LangChain4j facilitent l'intégration des LLM dans les projets Java, offrant des interfaces simplifiées pour des fonctionnalités avancées.
- **Développement d'agents** : Java est utilisé pour créer des agents intelligents, avec des frameworks tels que JACK Intelligent Agents, qui permettent de développer des systèmes multi-agents complexes.

2.4 Crew AI

Crew AI est un framework innovant conçu pour orchestrer des agents intelligents autonomes, facilitant leur collaboration pour accomplir des tâches complexes.

Caractéristiques principales

- **Modularité** : Permet l'intégration facile de différents types d'agents et de modèles de langage, offrant une flexibilité dans la conception des systèmes.



FIGURE 2.6 – Image de logo crew ai

- **Interface utilisateur intuitive** : Fournit des outils conviviaux pour la configuration et la gestion des agents, simplifiant ainsi le processus de développement.
- **Interopérabilité** : Compatible avec plusieurs langages de programmation et plateformes, facilitant l'intégration avec des systèmes existants.

Avantages pour le projet

- **Coordination efficace** : Crew AI assure une collaboration harmonieuse entre les agents, optimisant la répartition et l'exécution des tâches.
- **Suivi en temps réel** : Offre des fonctionnalités de monitoring en temps réel, permettant de superviser les performances des agents et d'ajuster les processus en conséquence.
- **Extensibilité** : Grâce à sa conception modulaire, Crew AI peut être étendu pour inclure de nouveaux agents ou fonctionnalités selon les besoins du projet.

Chapitre 3

Conception et Rôles des Agents Intelligents

Ce chapitre présente la démarche adoptée pour la réalisation de notre projet intitulé **Conversion d'Exigences en Logiciels Fonctionnels en Java avec LLM et Agents Intelligents**. Il expose les choix technologiques effectués, les outils utilisés ainsi que la conception des différents agents intelligents qui constituent le noyau de notre approche.

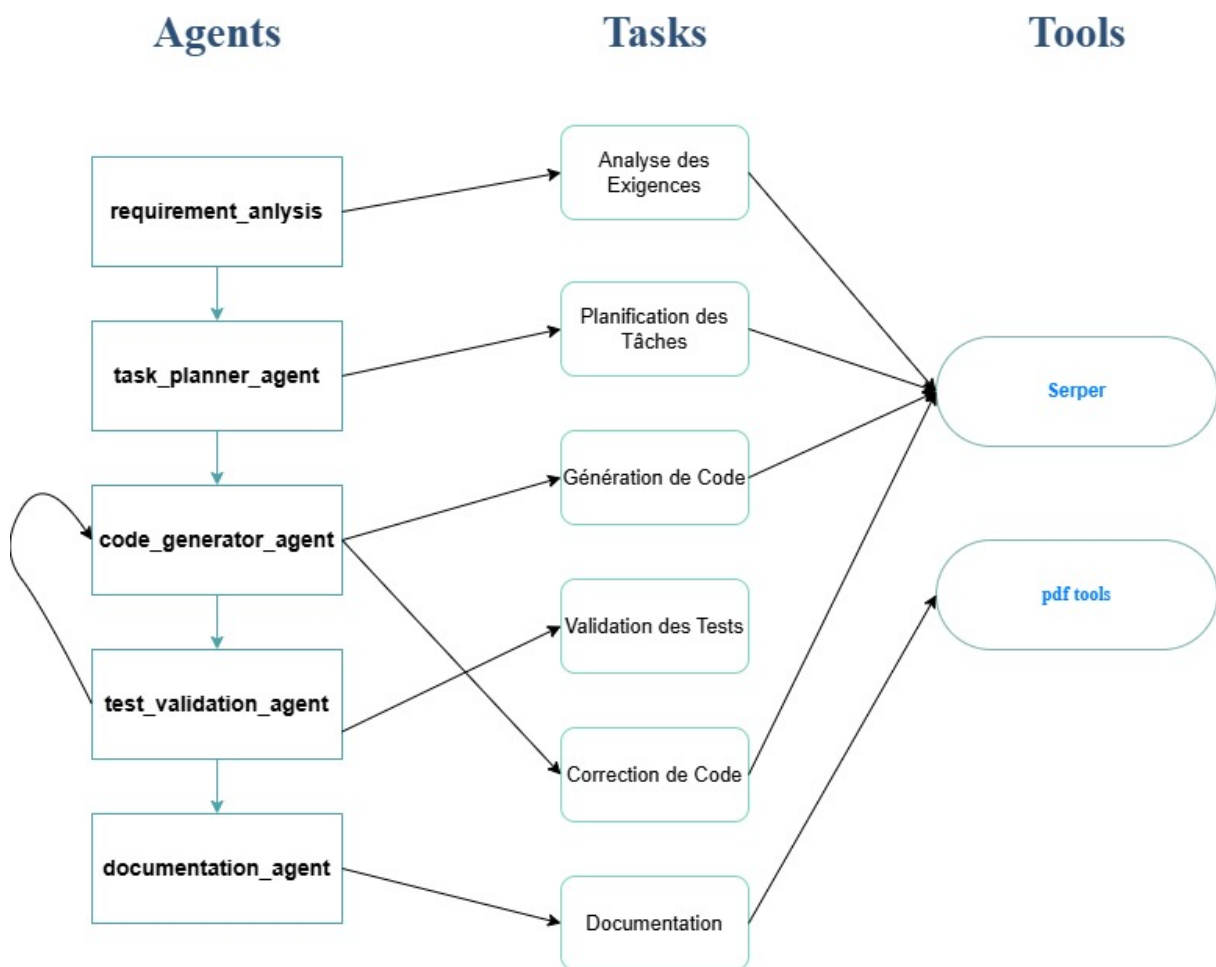


FIGURE 3.1 – Architecture du projet

Cette section détaille le fonctionnement des agents intelligents conçus pour notre projet ainsi que leurs interactions. Chaque agent joue un rôle spécifique dans le processus de

conversion des exigences en logiciels fonctionnels. Voici une explication complète basé sur le schéma figure 3.1.

3.1 Requirement Analysis Agent (Agent d'Analyse des Exigences)

Rôle : Cet agent analyse les besoins utilisateur exprimés en langage naturel et les convertit en exigences précises et utilisables. Il organise les exigences en :

- **Exigences fonctionnelles :** Ce que le système doit accomplir.
- **Exigences non fonctionnelles :** Performance, sécurité, fiabilité, etc.
- **Ambiguïtés et hypothèses :** Points nécessitant des clarifications ou des conditions implicites.

Sortie : Les spécifications résultantes sont transmises à l'Agent de Planification des Tâches.

3.2 Task Planner Agent (Agent de Planification des Tâches)

Rôle : Cet agent décompose les exigences analysées en un ensemble de tâches claires et structurées. Il fournit un plan détaillé avec :

- Les tâches principales et leurs dépendances.
- Les jalons essentiels pour suivre la progression.
- Des suggestions pour des itérations si nécessaire.

Sortie : La liste des tâches est transmise à l'Agent Générateur de Code.

3.3 Code Generator Agent (Agent Générateur de Code)

Rôle : Cet agent génère du code Java en fonction des tâches reçues. Il suit les meilleures pratiques, telles que la programmation orientée objet (POO), et produit :

- Du code modulaire et bien commenté.
- L'utilisation de bibliothèques standards si nécessaire.

En cas d'erreurs identifiées par l'Agent de Validation des Tests, cet agent peut aussi corriger les bugs, optimiser le code, et effectuer des ajustements nécessaires.

Sortie : Le code est transmis à l'Agent de Validation des Tests et Le code corrigé est renvoyé pour validation ou transmis à l'Agent de Documentation.

3.4 Test Validator Agent (Agent de Validation des Tests)

Rôle : Cet agent valide le code généré en exécutant des tests automatisés avec des outils comme JUnit. Il vérifie :

- La conformité du code avec les exigences fonctionnelles.
- La couverture des tests et les performances.
- Les éventuelles erreurs ou faiblesses à corriger.

Sortie : Si des erreurs sont détectées, un rapport est transmis à l'Agent de Correction de Code. Si tout est validé, le code passe à l'Agent de Documentation.

3.5 Documentation Agent (Agent de Documentation)

Rôle : Cet agent génère une documentation technique claire et complète du projet, incluant :

- Une explication des classes et méthodes.
- Les dépendances et exigences du projet.

- Des exemples d'utilisation et des guides.

Sortie : Une documentation en format PDF ou HTML est produite, accompagnant le logiciel final.

3.6 Coordination et Intégration des Outils

Le schéma met en évidence l'utilisation des outils suivants pour soutenir le pipeline :

- **API Serper :** Fournit des informations contextuelles pour enrichir l'analyse des exigences.
- **PDF Tools :** Génère des documents PDF pour la documentation technique.

3.7 Résumé du Flux Global

1. Analyse des exigences (Requirement Analysis Agent) - Génération des spécifications.
2. Planification des tâches (Task Planner Agent) - Liste structurée des tâches.
3. Génération de code (Code Generator Agent) - Code Java initial.
4. Validation des tests (Test Validator Agent) - Rapport de validation.
5. Correction de code (Code Fix Agent) - Code final sans erreurs.
6. Documentation (Documentation Agent) - Logiciel accompagné de sa documentation.

Cette coordination entre les agents garantit une transition fluide et une automatisation efficace du processus, assurant ainsi la qualité du produit final.

Chapitre 4

technologies utilisee

Le projet **Conversion d'exigences en logiciels fonctionnels en Java avec LLM et agents intelligents** a utilisé plusieurs technologies pour sa mise en œuvre. Parmi ces technologies, on trouve :

4.1 Choix du Modèle de Langage

Pour ce projet, nous avons choisi de travailler avec **Gemini Flash 1.5**, un modèle de langage avancé offrant une grande précision dans la compréhension et la génération de texte. L'accès à ce modèle a été assuré via une clé API, nous permettant d'intégrer ses capacités directement dans nos systèmes. Ce choix s'explique par les performances du modèle en matière d'analyse textuelle et de génération de contenu adapté aux exigences des logiciels.

4.2 API de recherche web

Pour les recherches web, nous avons utilisé l'API **Serper**, qui permet d'effectuer des requêtes sur le web et d'obtenir des résultats en temps réel. Grâce à cette intégration, nous avons pu enrichir nos analyses avec des données contextuelles pertinentes et actualisées, facilitant ainsi la compréhension des exigences et l'amélioration du processus de génération de code.



FIGURE 4.1 – Serper logo

4.3 Python

Python a été choisi pour la programmation principale de notre projet. Il a servi à orchestrer l'exécution des agents et à interagir avec les modèles de traitement du langage naturel pour analyser les exigences.

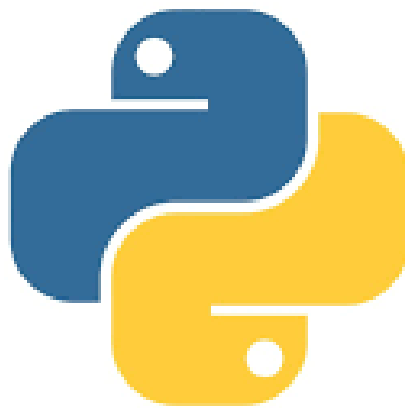


FIGURE 4.2 – Python Logo

4.4 Réalisation d'interface en utilisant Flask

Flask est un micro-framework web en Python, léger et flexible, permettant de développer des applications web avec une structure simple et efficace. Dans notre projet, nous avons utilisé Flask pour faciliter l'interaction de l'utilisateur avec le système, en lui offrant une interface intuitive où il peut :

- Saisir ses exigences en langage naturel,
- Visualiser les résultats générés par chaque agent en temps réel,
- Suivre l'évolution du processus (de l'analyse des exigences jusqu'à la génération du code et la documentation).

Grâce à Flask, nous avons pu structurer une interface web accessible, permettant une communication fluide entre l'utilisateur et les agents intelligents tout en assurant une expérience utilisateur optimisée.



FIGURE 4.3 – Flask Logo

Chapitre 5

Implémentation

Le système vise à automatiser la conversion d'exigences utilisateur en code Java fonctionnel à travers une chaîne d'agents intelligents. L'implémentation repose sur une architecture modulaire utilisant Flask comme framework web et CrewAI pour l'orchestration des agents.

5.1 Configuration des clés API

5.1.1 Création et Configuration de la clé API Google Gemini

Pour utiliser le modèle Gemini Flash dans notre projet, nous avons généré une clé API Google via la Google AI Studio. Le processus s'est déroulé en plusieurs étapes :

1. Accès à la plateforme Google AI Studio [1] et connexion avec un compte Google.
2. Activation de l'API Gemini en créant un projet sur Google Cloud Console.
3. Génération d'une clé API privée, qui nous permet d'envoyer des requêtes aux services de Gemini.
4. Intégration de la clé dans notre application .

Cette clé API nous permet d'accéder aux fonctionnalités avancées de Gemini Flash, notamment pour l'analyse des exigences et la génération de code Java, tout en assurant un usage sécurisé et conforme aux bonnes pratiques de gestion des clés API.

```
## call the gemini models
llm=ChatGoogleGenerativeAI(model="gemini-1.5-flash",
                             verbose=True,
                             temperature=0.5,
                             google_api_key="AIzaSyAQA15yQUBeAwkJJcXbcOSThCwaEh_p_IY")
```

FIGURE 5.1 – Configuration de cle api Gemini

5.1.2 Création et Configuration de la Clé API Serper

Pour intégrer l'API Serper dans notre projet, on a commencé par créer un compte sur la plateforme Serper.dev. Une fois connectés, on a généré une clé API unique à partir du tableau de bord de l'API. Afin de sécuriser cette clé et éviter son exposition dans le code, on l'a stockée sous forme de variable d'environnement dans un fichier `tools`.

Cette clé nous permet d'effectuer des requêtes à Serper afin de récupérer des informations en temps réel et enrichir l'analyse des exigences. L'intégration a été réalisée en envoyant des requêtes HTTP via `requests`, avec l'authentification assurée par l'en-tête `X-API-KEY`.

Grâce à cette configuration, notre système peut exploiter des données contextuelles issues du web tout en respectant les bonnes pratiques de gestion des clés API.

```
from dotenv import load_dotenv
load_dotenv()
import os
import traceback

os.environ['SERPER_API_KEY'] = "b67c8ee924e002d147b2f936308e80e54909539c"

from crewai_tools import SerperDevTool

# Initialize the tool for internet searching capabilities
tool = SerperDevTool()
```

FIGURE 5.2 – Configuration de cle api Serper

5.2 Implémentation des Agents

Dans le cadre de ce projet, plusieurs agents intelligents ont été implémentés pour assurer l'exécution efficace des différentes étapes du processus, allant de l'analyse des exigences jusqu'à la documentation du code généré. Chaque agent a un rôle spécifique et interagit avec les autres agents pour accomplir des tâches précises.

Les agents suivants ont été développés et intégrés dans le système :

Agent d'Analyse des Exigences Cet agent est conçu pour analyser les exigences fournies en langage naturel. Il identifie et structure les besoins fonctionnels et non fonctionnels pour les transmettre à l'agent de planification.

```
requirement_analysis = Agent(
    role='Requirement Analyst for Java Code Generation',
    goal='Analyze and extract key requirements from natural language to generate Java source code.',
    backstory=(
        'You are a Requirement Analyst specializing in converting natural language into precise Java software requirements. '
        'Organize your analysis into the following sections:\n'
        '1. Functional Requirements\n'
        '2. Non-Functional Requirements\n'
        '3. Assumptions\n'
        '4. Ambiguities\n'
        'Ensure your output is structured and concise.'
    ),
    tools=[tool],
    verbose=True,
    llm=llm,
    max_rpm=None,
)
```

FIGURE 5.3 – Agent d'Analyse des Exigences

Agent de Planification L'agent de planification transforme les exigences structurées en un ensemble de tâches ordonnées, prêtes à être réalisées. Il définit également les dépendances et les jalons essentiels.

```
task_planner_agent = Agent(
    role='Task Planner and Decomposer',
    goal="Plan and decompose the project requirements into actionable tasks for Java application development.",
    backstory=(
        "You are a software project manager specialized in agile development. "
        "You excel at analyzing software requirements and breaking them into clear, manageable tasks. "
        "Your expertise ensures efficient project planning and organization to guide developers effectively."
    ),
    tools=[tool],
    verbose=True,
    llm=llm
)
```

FIGURE 5.4 – Agent de Planification

Agent Générateur de Code Cet agent prend en charge les tâches planifiées et produit du code Java respectant les spécifications fonctionnelles et les bonnes pratiques de développement logiciel.

```
code_generator_agent = Agent(
    role='Code Generator Agent',
    goal='Generate clean, efficient, and functional Java source code based on specified tasks and user requirements, '
        'ensuring adherence to best practices.',
    backstory=(
        "The Code Generator Agent is a sophisticated AI tool trained on a wide variety of programming paradigms, "
        "including object-oriented programming, functional programming, and more. It was designed to interpret user "
        "requirements in natural language and translate them into high-quality Java source code. With its advanced LLM "
        "model, the agent is capable of understanding complex programming tasks, identifying potential issues, "
        "and ensuring "
        "the generated code adheres to industry standards. The agent is particularly focused on producing modular, "
        "scalable,"
        "and well-documented code, aiming to accelerate development while reducing errors and improving "
        "maintainability."
    ),
    tools=[tool],
    verbose=True,
    llm=llm
)
```

FIGURE 5.5 – Agent générateur du code

Agent de Test et Validation Une fois le code généré, cet agent effectue des tests pour valider sa conformité aux exigences initiales. Il génère également des rapports détaillés en cas d'erreurs.

```
test_validation_agent = Agent(
    role='Test Validator Agent',
    goal="Validate the correctness, efficiency, and maintainability of the generated Java code by writing and executing JUnit test cases.",
    backstory=(
        "You are a Test Validator Agent with expertise in validating software code using JUnit. "
        "Your primary role is to ensure that the generated Java code is correct, efficient, "
        "and meets the requirements specified in the planning phase. You write, execute, and validate test cases "
        "using JUnit to ensure that all functionalities are properly tested and implemented. You also evaluate the "
        "code for adherence to industry standards, performance bottlenecks, and maintainability. "
        "Provide detailed test case results and suggest fixes for any issues identified."
    ),
    tools=[],
    verbose=True,
    llm=llm
)
```

FIGURE 5.6 – Agent de test et validation

Agent Documentation Cet agent finalise le processus en générant une documentation technique détaillée qui accompagne le logiciel, incluant des explications sur les classes, les méthodes et les exemples d'utilisation.

```
documentation_agent = DocumentationAgent(
    role='Documentation Agent',
    goal='Take the generated code and produce a comprehensive report explaining the role of each class and method, then convert the documentation into a PDF file.',
    backstory=(
        "You are a Documentation Agent skilled in analyzing Java source code and generating clear, concise documentation. "
        "Your primary responsibility is to explain the purpose of each class and method, detailing their inputs, outputs, and relationships. "
        "You then convert this structured documentation into a PDF file for easy sharing."
    ),
    tools=[pdf_tool], # Utiliser l'outil PDF défini ci-dessus
    verbose=True,
    llm=llm
)
```

FIGURE 5.7 – Agent Documentation

5.3 Processus Séquentiel dans app.py

Dans le cadre de ce projet, les interactions entre les différents agents suivent un processus séquentiel orchestré dans le fichier `app.py`. Chaque agent accomplit une tâche spécifique et transmet ses résultats à l'agent suivant, garantissant ainsi une coordination fluide et une progression logique tout au long du pipeline.

Dans le fichier `app.py`, ce processus séquentiel est implémenté en utilisant des fonctions spécifiques pour chaque étape. Les résultats produits par un agent deviennent les entrées du suivant, garantissant une traçabilité complète et une exécution cohérente. Voici un exemple simplifié :

-Cette image illustre l'appel de l'agent d'analyse des exigences. Cet agent reçoit les données utilisateur et produit des spécifications structurées, qui seront utilisées dans les étapes suivantes.

```
try:
    # 1. Requirements Analysis
    crew_analysis = Crew(
        agents=[requirement_analysis],
        tasks=[RequirementAnalysis.req(topic)],
        process=Process.sequential,
    )
    analysis_result = crew_analysis.kickoff()
    print("Analysis result:", analysis_result) # Debug log

    if isinstance(analysis_result, dict):
        formatted_analysis = json.dumps(analysis_result, indent=2)
    else:
        formatted_analysis = str(analysis_result)
    results['data']['requirements'] = formatted_analysis
```

FIGURE 5.8 – Appel de l'agent requirement

-Cette image montre l'agent de planification des tâches en action. Il décompose les spécifications en tâches détaillées et ordonnées, prêtes pour l'étape de génération de code.

```
crew_planning = Crew(  
    agents=[task_planner_agent],  
    tasks=[TaskPlanning.plan_and_decompose(inputs['topic'],formatted_analysis_result)],  
    process=Process.sequential,  
)  
  
print("\n=== Lancement de la Planification et Décomposition des Tâches ===\n")  
planning_result = crew_planning.kickoff(formatted_analysis_result)  
  
print("\n=== Résultats de la Planification ===\n")  
if isinstance(planning_result, dict):  
    for key, value in planning_result.items():  
        print(f"{key}:\n{value}\n{'-' * 50}")  
else:  
    print(planning_result)
```

FIGURE 5.9 – Appel de l'agent planification

-L'agent générateur de code est activé pour transformer les tâches en code source Java. L'image présente le flux de cette étape critique dans le processus

```
crew_generation = Crew(  
    agents=[code_generator_agent],  
    tasks=[CodeGenerationTask.code_generation(inputs['topic'], planning_result)],  
    process=Process.sequential,  
)  
print("=== lancement de code_generation_agent ===\n")  
  
code_result = crew_generation.kickoff()  
print("\n##### Code #####")  
print(code_result)
```

FIGURE 5.10 – Appel de l'agentgénérateurur de code

-Après la génération du code, cet agent effectue des tests automatisés pour valider sa conformité aux exigences et détecter les éventuelles erreurs.

```

crew_test_validation = Crew(
    agents=[test_validation_agent],
    tasks=[TestValidationTask.validate_code(inputs['topic'], code_result)], # Passer le code généré
    process=Process.sequential,
)

print("\n=== Lancement de la Validation des Tests ===\n")
validation_result = crew_test_validation.kickoff()

# Afficher les résultats de la validation des tests
print("\n##### Résultats de la Validation des Tests #####")
if isinstance(validation_result, dict):
    for key, value in validation_result.items():
        print(f"{key}: \n{value}\n{'-' * 50}")
else:
    print(validation_result)

```

FIGURE 5.11 – Appel de l’agent de test

-Dans le cas où des erreurs sont détectées lors de la validation, l’agent générateur de code est réinvité à corriger le code, garantissant une version fonctionnelle.

```

#####
validation_status = TestValidationTask.extract_final_status(validation_result)

if validation_status and validation_status.lower() != 'valid':

    fixed_crew = Crew(
        agents=[code_generator_agent],
        tasks=[CodeFixTask.fix_code(inputs['topic'], code_result, validation_result)],
        process=Process.sequential )

    code_result = fixed_crew.kickoff()
    print("\n=== Lancement de la Fixation des Codes ===\n")
    print(code_result)
####

```

FIGURE 5.12 – Appel 2 de l’agent générateur de code si le premier non valider

-Cette dernière étape illustre l’agent de documentation générant des fichiers clairs et détaillés pour accompagner le logiciel livré.

```

print("\n=== Lancement de la Documentation ===\n")

documentation = documentation_agent.generate_documentation(code_result,inputs['topic'])

# Afficher ou traiter la documentation
print("\n=== Documentation Générée ===\n")
print(documentation)

```

FIGURE 5.13 – appel de l’agent documentation

Grâce à cette organisation séquentielle, le système offre une structure claire et extensible, permettant d’ajouter ou de modifier des étapes au besoin. Ce processus garantit également que chaque agent collabore efficacement pour produire un compte rendu final complet et précis, incluant le code généré et la documentation.

5.4 Traitement Intermédiaire : Création des Tâches et des Outils

Entre la création des agents et leur appel dans le processus séquentiel, des étapes supplémentaires sont mises en place pour garantir des résultats optimaux. Ces étapes intermédiaires incluent la création des tâches et des outils, deux composants essentiels pour chaque agent dans notre système.

5.4.1 Création des Tâches (`task.py`)

Avant que chaque agent ne puisse effectuer sa mission, des tâches doivent être créées pour chaque étape du processus. Le fichier `task.py` est responsable de la gestion de ces tâches, qui sont structurées et attribuées aux agents en fonction de leurs rôles spécifiques. Ces tâches définissent les actions précises que chaque agent doit réaliser et assurent une coordination fluide entre les différentes étapes du pipeline.

Chaque agent reçoit une ou plusieurs tâches définies dans `task.py`, en fonction de l'étape dans laquelle il intervient. Par exemple, l'agent d'analyse des exigences est responsable de la transformation des données brutes en spécifications structurées, tandis que l'agent de planification décompose ces spécifications en tâches précises.

5.4.2 Création des Outils (`tools.py`)

Les outils sont également nécessaires pour permettre aux agents de remplir leurs fonctions efficacement. Ces outils, définis dans le fichier `tools.py`, sont des fonctions ou des services externes qui aident les agents à accomplir leurs tâches. Par exemple, pour l'agent d'analyse des exigences, un outil pourrait être un modèle de traitement du langage naturel qui extrait les entités clés des exigences. Pour l'agent générateur de code, un outil pourrait être une bibliothèque ou une API permettant de convertir des tâches planifiées en code Java.

Les outils sont configurés dans `tools.py` et sont appelés par les agents lors de leur exécution. Chaque agent utilise les outils qui correspondent à son rôle, garantissant ainsi que les résultats sont de qualité et que le flux du processus est optimisé.

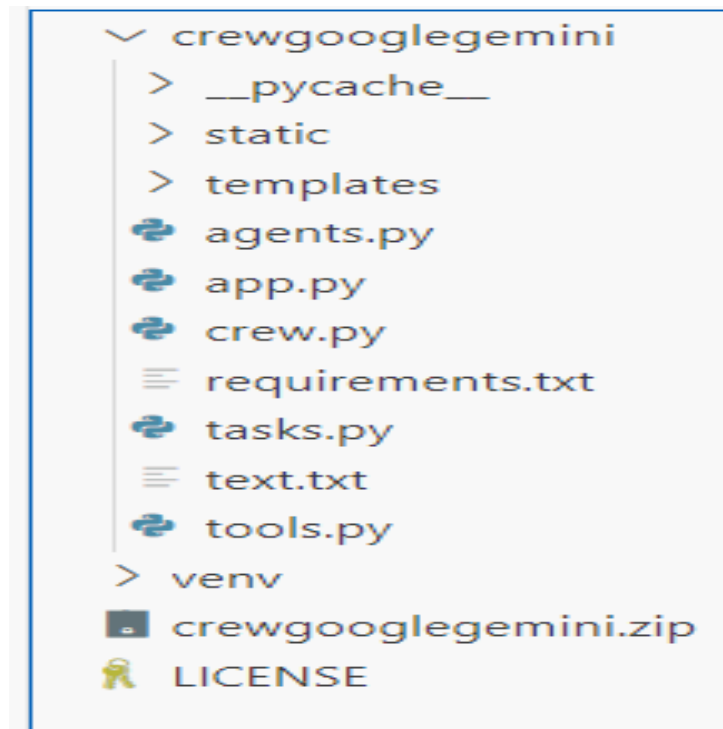


FIGURE 5.14 –

5.4.3 Importance de ces Étapes Intermédiaires

Ces étapes intermédiaires sont cruciales pour le bon fonctionnement du système. La création des tâches assure que chaque agent reçoit une entrée clairement définie et structurée, tandis que la configuration des outils garantit que chaque agent dispose des ressources nécessaires pour accomplir ses objectifs. Ensemble, ces processus permettent de maintenir une coordination fluide et une exécution cohérente, garantissant ainsi un produit final fonctionnel et bien documenté.

5.5 Développement de l'interface utilisateur

Pour ce projet, comme mentionné précédemment, l'interface utilisateur a été développée en utilisant Flask, un micro-framework Python léger et flexible.

A screenshot of a code editor window titled 'app.py'. The code is written in Python and uses Flask. It imports Flask, render_template, request, and jsonify. It creates an app object with Flask(__name__). It defines an index route that renders 'index.html'. It also defines a generate route that takes a POST request, checks for a 'topic' parameter, and returns a JSON error if it's missing. The code is as follows:

```
from flask import Flask, render_template, request, jsonify
app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/generate', methods=['POST'])
def generate():
    topic = request.form.get('topic')
    if not topic:
        return jsonify({'error': 'Topic is required'}), 400
    # ... suite du code ...
```

FIGURE 5.15 – Configuration Flask

Chapitre 6

Résultats de l'Application

Pour lancer l'application et afficher l'interface créée, il suffit d'exécuter le fichier `app.py`. Ce fichier sert de point d'entrée pour le système et orchestre l'exécution des différents agents. Une fois `app.py` lancé, l'interface web est déployée, permettant aux utilisateurs d'interagir avec l'application et de visualiser les résultats générés par les agents à chaque étape du processus.



```
DEBUG MODE: on
2025-01-17 12:35:203 - _internal.py-_internal:97 - INFO: WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
2025-01-17 12:23:35,203 - 11240 - _internal.py-_internal:97 - INFO: Press CTRL+C to quit
2025-01-17 12:23:35,261 - 11240 - _internal.py-_internal:97 - INFO: * Restarting with watchdog (windowsapi)
c:\Users\ALSAB\Desktop\CrewAI-Projects-main\venv\lib\site-packages\pydantic\_internal\generate_schema.py:777: UserWarning: Mixing V1 models and V2 models (or constructs, like `TypeAdapter`) is not supported. Please upgrade `CrewAgentExecutor` to V2.
warn(
c:\Users\ALSAB\Desktop\CrewAI-Projects-main\venv\lib\site-packages\pydantic\_internal\generate_schema.py:777: UserWarning: Mixing V1 models and V2 models (or constructs, like `TypeAdapter`) is not supported. Please upgrade `CrewAgentExecutor` to V2.
warn(
2025-01-17 12:23:49,218 - 4352 - _internal.py-_internal:97 - WARNING: * Debugger is active!
2025-01-17 12:23:49,241 - 4352 - _internal.py-_internal:97 - INFO: * Debugger PIN: 537-552-685
```

FIGURE 6.1 – le lien 127.0.0.1 :5000

Après l'exécution de l'application, il suffit de se rendre sur le lien suivant pour accéder à l'interface : 127.0.0.1:5000. L'interface qui s'affiche.



FIGURE 6.2 – L'interface de l'application

6.1 Tests de l'application

Après avoir lancé l'application, nous allons tester deux exemples différents pour illustrer le fonctionnement du système et la manière dont il génère et valide le code.

6.1.1 Exemple 1 : application de calculatrice

Dans ce premier exemple, nous demandons à l'application de réaliser une application de calculatrice simple. Après l'exécution du processus, l'application génère directement le code Java correspondant, qui est ensuite validé avec succès. Ce cas montre que le système peut correctement générer un code fonctionnel à partir des exigences et valider celui-ci sans besoin de corrections supplémentaires.



FIGURE 6.3 – En lançant la génération de code

Résultats du premier agent : Analyse des exigences

Le premier agent analyse les exigences en langage naturel fournies par l'utilisateur et les transforme en spécifications structurées.

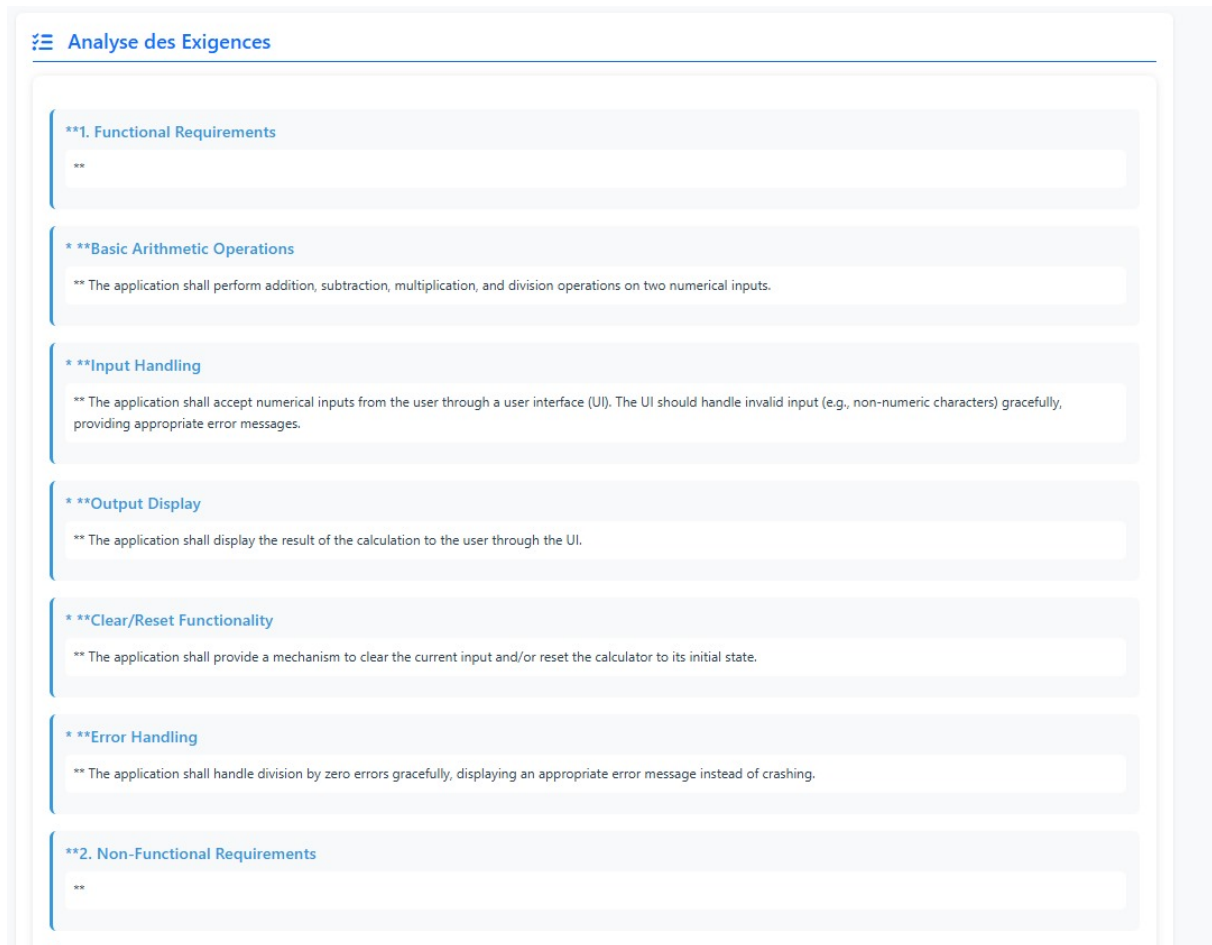


FIGURE 6.4 – Captures des résultats d'analyses des exigences

Résultats du deuxième agent : Planification des tâches

Le deuxième agent prend les spécifications structurées et les transforme en un ensemble de tâches planifiées, prêtes à être exécutées.

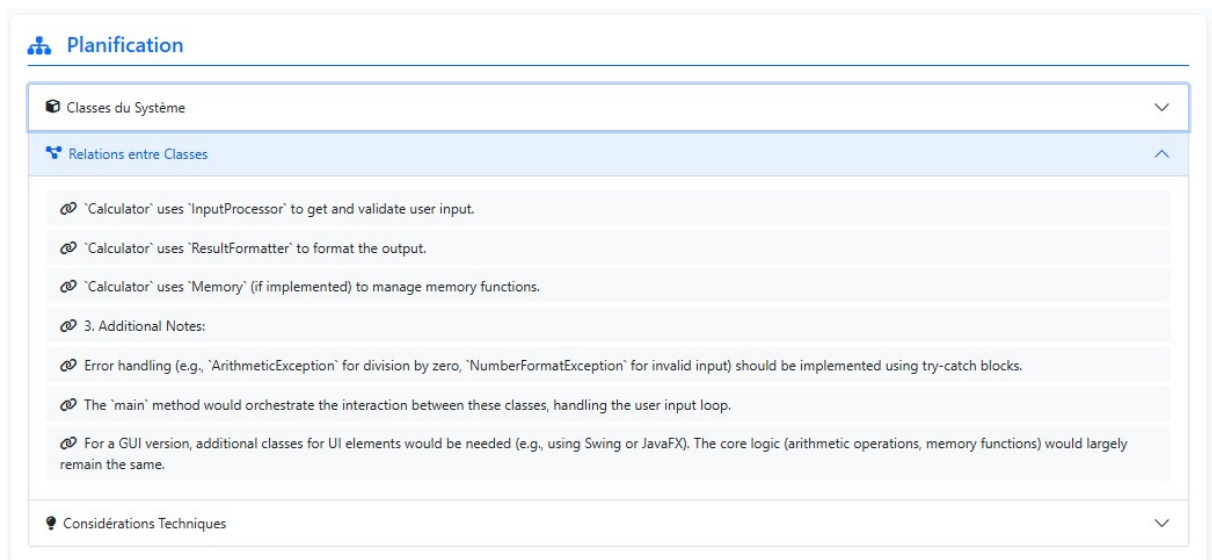


FIGURE 6.5 – Captures des résultats de planifications

Résultats du troisième agent : Génération du code

Le troisième agent génère le code Java basé sur les tâches planifiées et les spécifications. Voici un aperçu du code généré pour l'application de calculatrice.

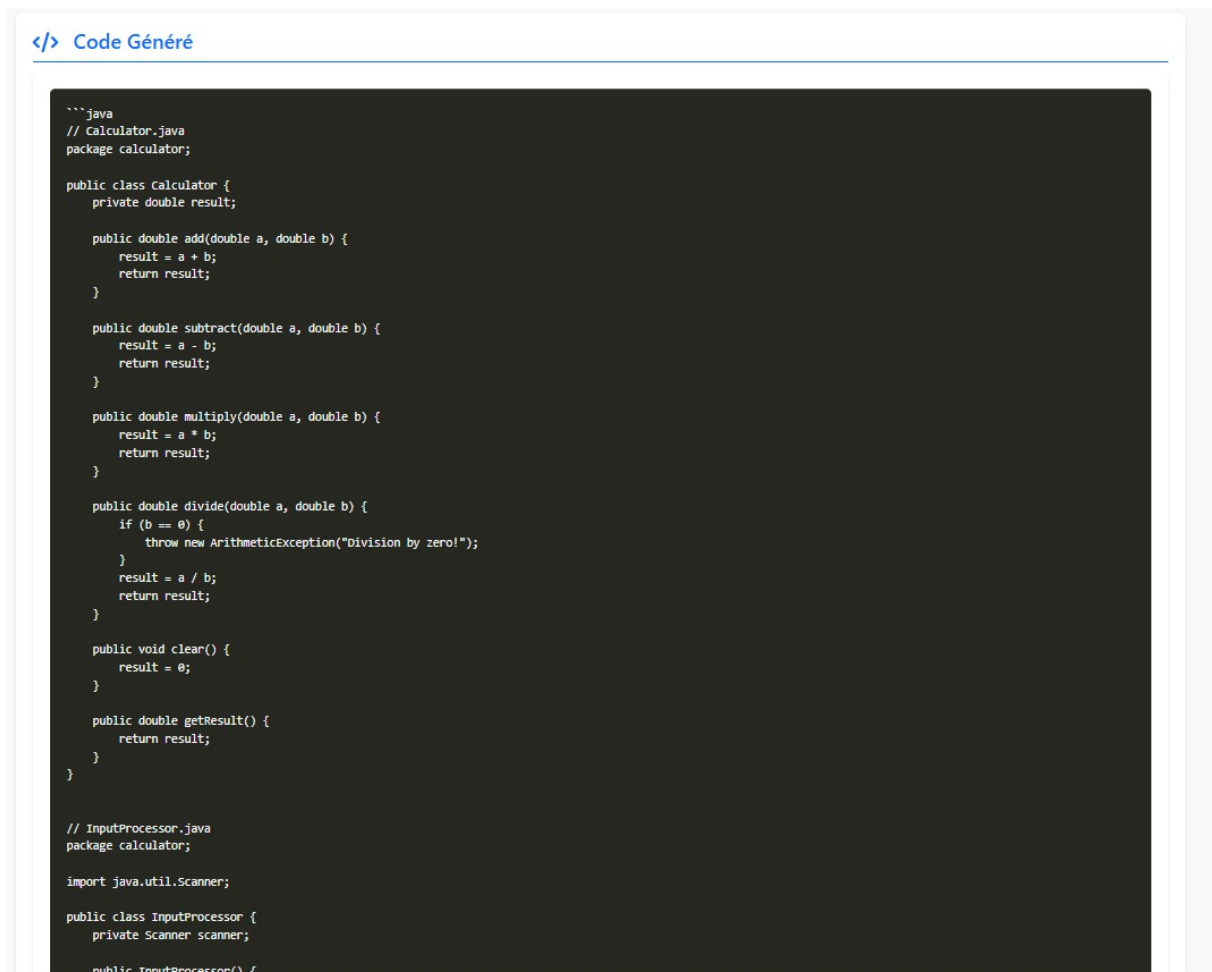


FIGURE 6.6 – Captures des résultats de code generer

Résultats du quatrième agent : Validation du code

Après génération, le quatrième agent valide le code. Dans cet exemple, le code est validé automatiquement sans nécessiter de corrections supplémentaires.

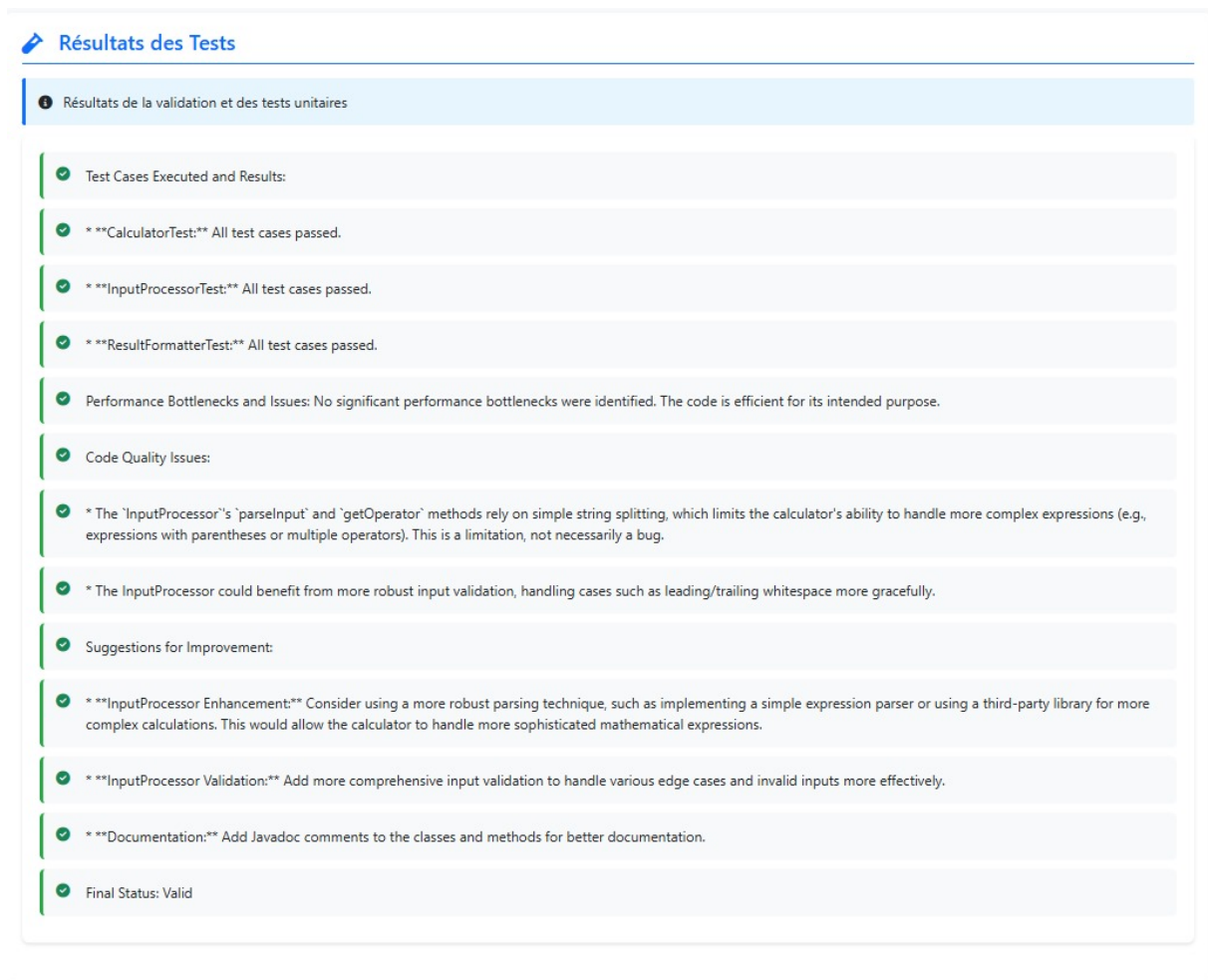


FIGURE 6.7 – Capture d'écran : Code généré pour l'application de calculatrice validé automatiquement

Documentation générée

Enfin, l'agent de documentation génère une première page de la documentation technique qui accompagne le code généré, incluant des explications sur les classes et les méthodes.

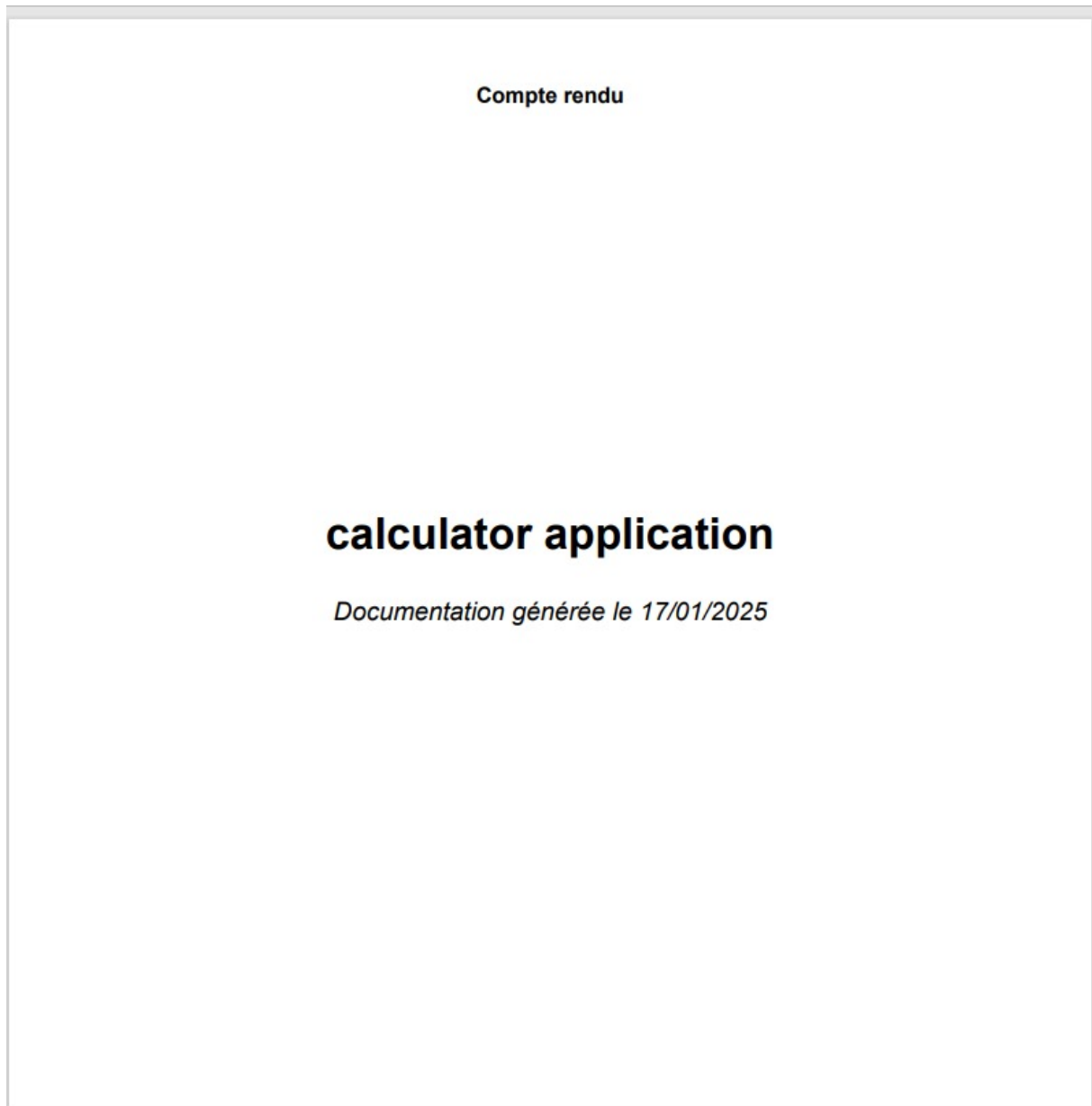


FIGURE 6.8 – Capture d'écran : première page de code générée.

6.1.2 Exemple 2 : Système de gestion des tâches

Dans le deuxième exemple, nous demandons à l'application de réaliser un système de gestion des tâches. Après l'exécution du processus, l'application génère le code Java correspondant. Cependant, dans ce cas, le code généré n'est pas validé directement car il contient des erreurs. L'agent de test et validation détecte ces erreurs et demande à l'agent générateur de code de corriger le code. Une fois la correction effectuée, le code est à nouveau testé et validé avec succès.

Résultats du premier agent : Analyse des exigences

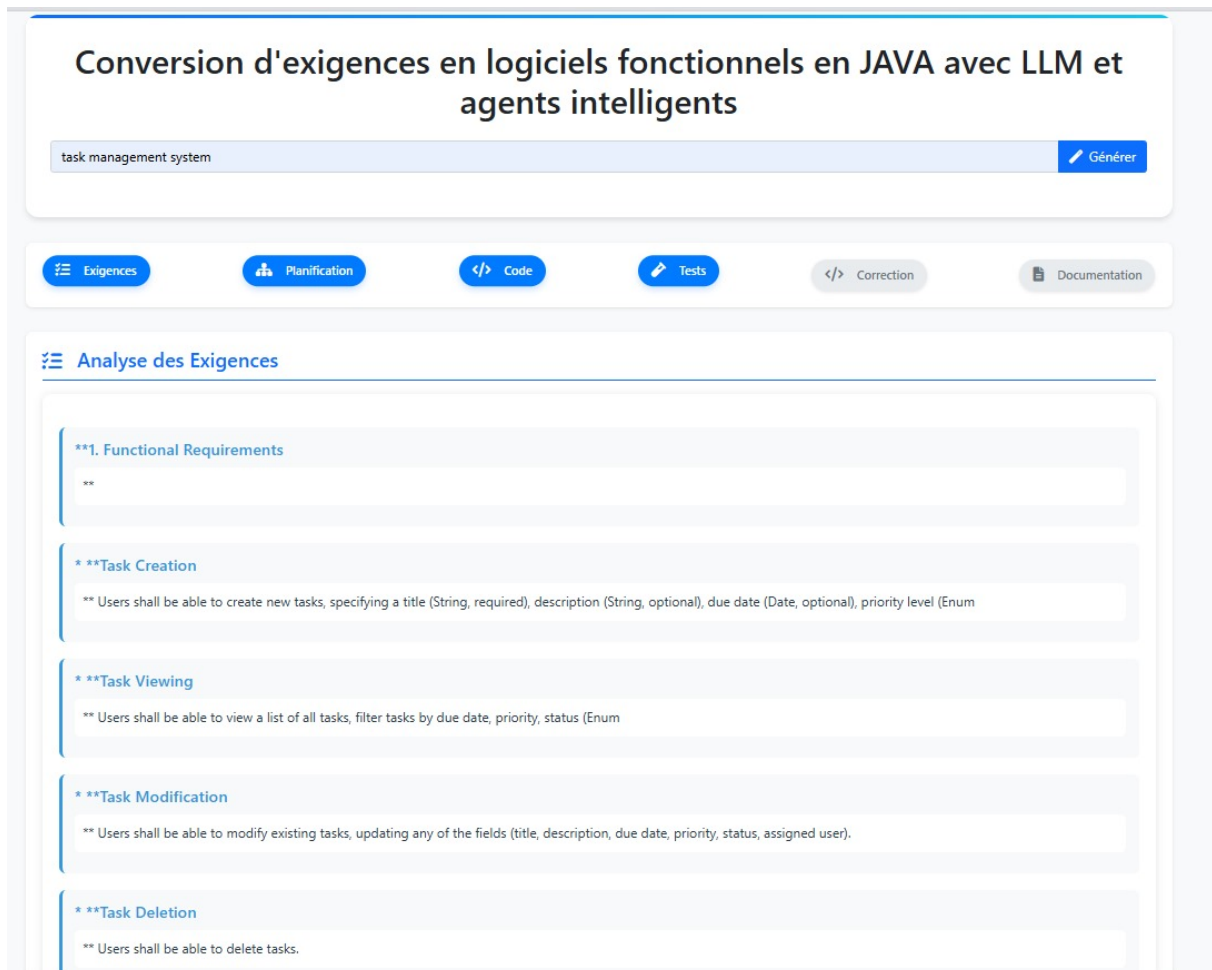


FIGURE 6.9 – Captures des résultats d'analyse des exigences

Résultats du deuxième agent : planification des tâches

Planification

Classes du Système

Relations entre Classes

One-to-many relationship between 'User' and 'Task'. A 'User' can have many 'Tasks' assigned to them. This could be implemented using a 'List' in the 'User' class, or using a foreign key in the 'Task' table linking to the 'User' table in a database.

Considérations Techniques

Considération 1 Exception Handling:** Appropriate exception handling should be implemented in all methods.	Considération 2 Input Validation:** Input validation should be performed to prevent invalid data from being stored.	Considération 3 Security:** Secure authentication and authorization mechanisms need to be implemented (e.g., using Spring Security).
Considération 4 UI:** The UI design and implementation are not detailed here, but would be a significant part of the project.	Considération 5 Reporting:** The specific reporting requirements need to be clarified and implemented (e.g., using reporting libraries or custom solutions).	Considération 6 Testing:** Thorough testing is crucial for ensuring the quality and reliability of the application.

fig :enter-label

Résultats du troisième agent : génération du code

</> Code Généré

```
```java
import java.util.Date;
import java.util.List;

enum Priority {
 HIGH, MEDIUM, LOW
}

enum Status {
 OPEN, IN_PROGRESS, COMPLETED
}

class Task {
 private Long id;
 private String title;
 private String description;
 private Date dueDate;
 private Priority priority;
 private Status status;
 private String assignedUser;

 public Task(String title) {
 if (title == null || title.isEmpty()) {
 throw new IllegalArgumentException("Title cannot be null or empty.");
 }
 this.title = title;
 this.priority = Priority.MEDIUM;
 this.status = Status.OPEN;
 }

 public Long getId() {
 return id;
 }

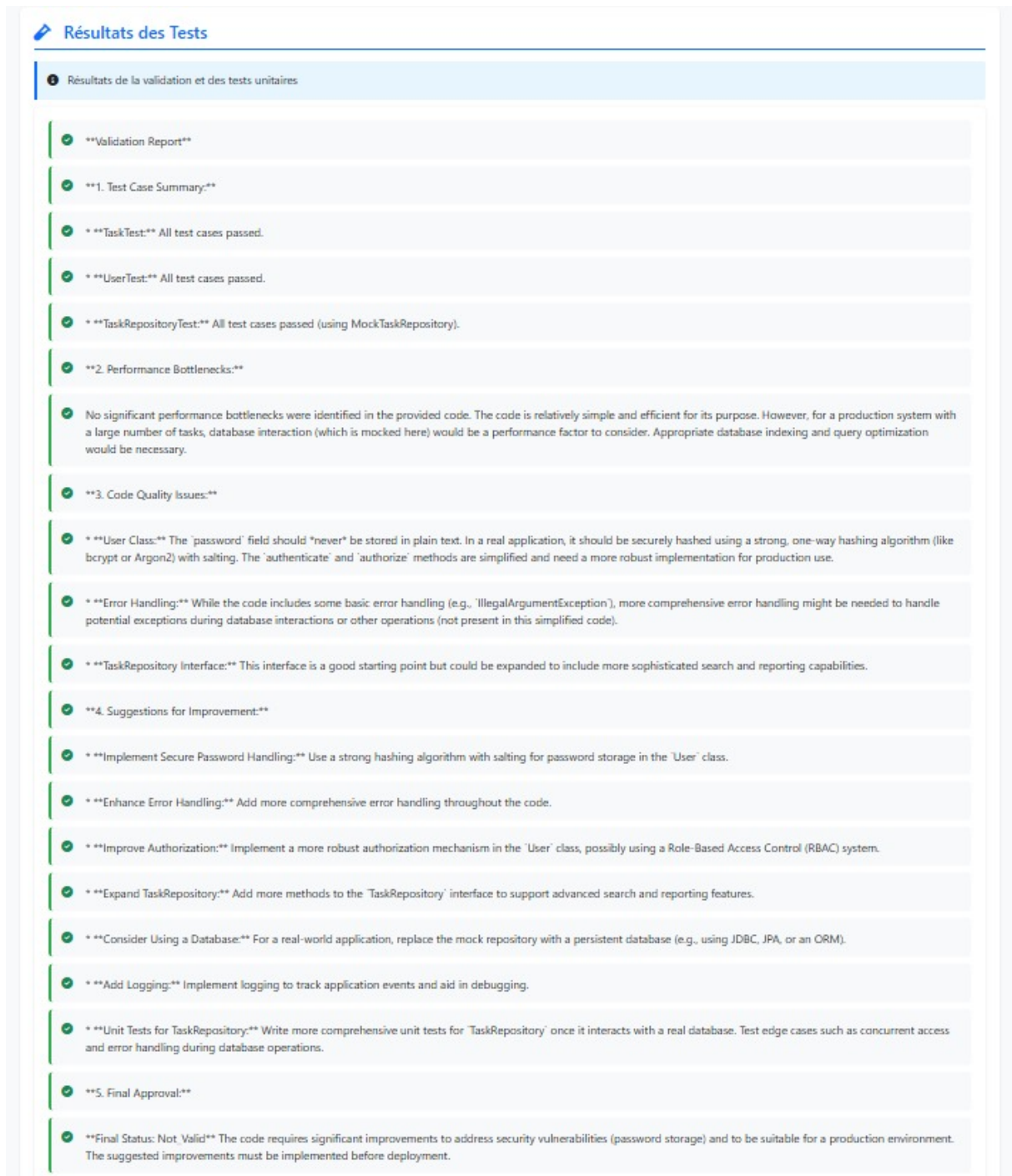
 public void setId(Long id) {
 this.id = id;
 }

 public String getTitle() {
 return title;
 }

 public void setTitle(String title) {
 if (title == null || title.isEmpty()) {
 throw new IllegalArgumentException("Title cannot be null or empty.");
 }
 this.title = title;
 }
}
```

FIGURE 6.11 – Capture d'une partie de code generer

## Résultats du quatrième agent : Validation du code



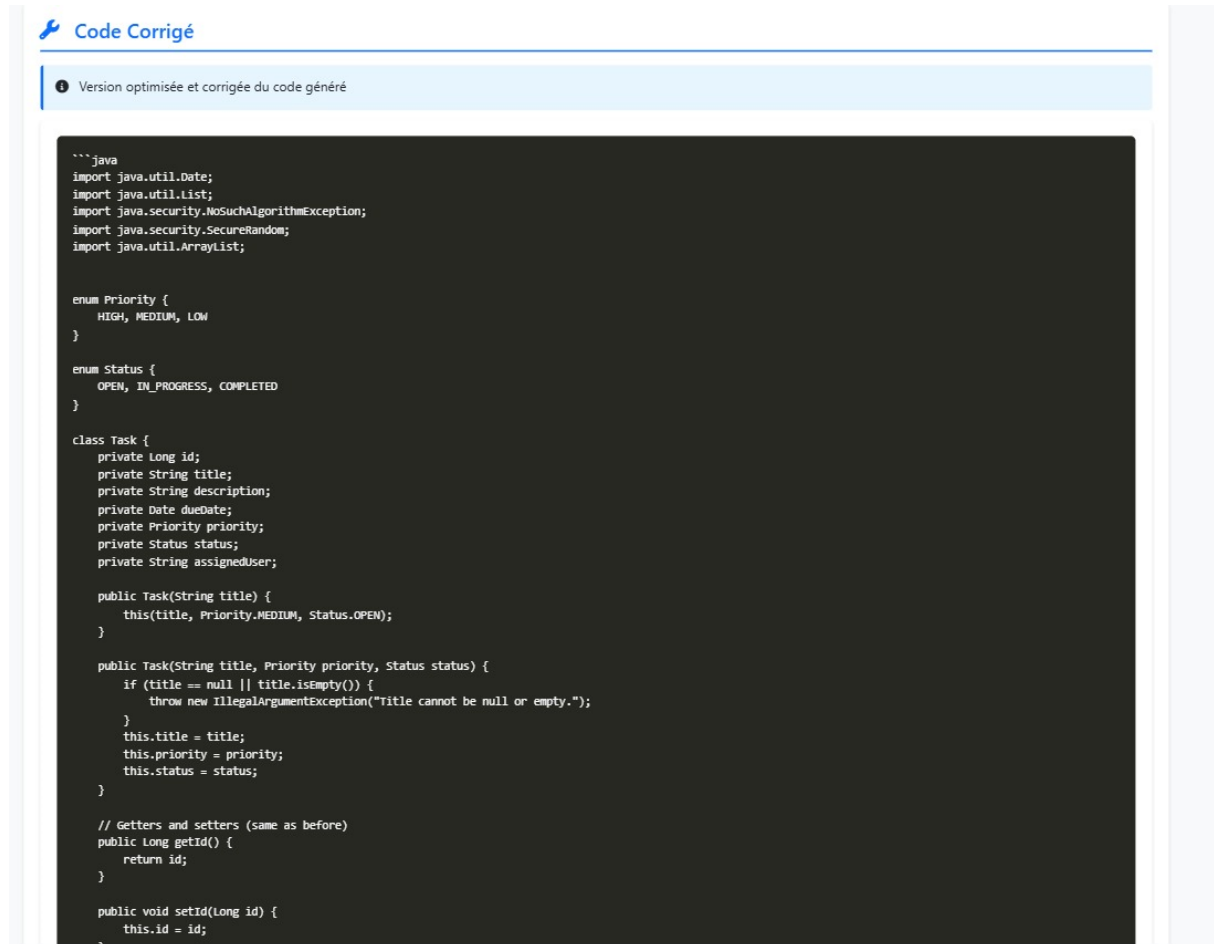
**Résultats des Tests**

**Résultats de la validation et des tests unitaires**

- ✓ **\*\*Validation Report\*\***
- ✓ **\*\*1. Test Case Summary:\*\***
- ✓ **\* \*\*TaskTest:\*\* All test cases passed.**
- ✓ **\* \*\*UserTest:\*\* All test cases passed.**
- ✓ **\* \*\*TaskRepositoryTest:\*\* All test cases passed (using MockTaskRepository).**
- ✓ **\*\*2. Performance Bottlenecks:\*\***
- ✓ No significant performance bottlenecks were identified in the provided code. The code is relatively simple and efficient for its purpose. However, for a production system with a large number of tasks, database interaction (which is mocked here) would be a performance factor to consider. Appropriate database indexing and query optimization would be necessary.
- ✓ **\*\*3. Code Quality Issues:\*\***
- ✓ **\* \*\*User Class:\*\* The 'password' field should 'never' be stored in plain text. In a real application, it should be securely hashed using a strong, one-way hashing algorithm (like bcrypt or Argon2) with salting. The 'authenticate' and 'authorize' methods are simplified and need a more robust implementation for production use.**
- ✓ **\* \*\*Error Handling:\*\* While the code includes some basic error handling (e.g., 'IllegalArgumentException'), more comprehensive error handling might be needed to handle potential exceptions during database interactions or other operations (not present in this simplified code).**
- ✓ **\* \*\*TaskRepository Interface:\*\* This interface is a good starting point but could be expanded to include more sophisticated search and reporting capabilities.**
- ✓ **\*\*4. Suggestions for Improvement:\*\***
- ✓ **\* \*\*Implement Secure Password Handling:\*\* Use a strong hashing algorithm with salting for password storage in the 'User' class.**
- ✓ **\* \*\*Enhance Error Handling:\*\* Add more comprehensive error handling throughout the code.**
- ✓ **\* \*\*Improve Authorization:\*\* Implement a more robust authorization mechanism in the 'User' class, possibly using a Role-Based Access Control (RBAC) system.**
- ✓ **\* \*\*Expand TaskRepository:\*\* Add more methods to the 'TaskRepository' interface to support advanced search and reporting features.**
- ✓ **\* \*\*Consider Using a Database:\*\* For a real-world application, replace the mock repository with a persistent database (e.g., using JDBC, JPA, or an ORM).**
- ✓ **\* \*\*Add Logging:\*\* Implement logging to track application events and aid in debugging.**
- ✓ **\* \*\*Unit Tests for TaskRepository:\*\* Write more comprehensive unit tests for 'TaskRepository' once it interacts with a real database. Test edge cases such as concurrent access and error handling during database operations.**
- ✓ **\*\*5. Final Approval:\*\***
- ✓ **\*\*Final Status: Not Valid\*\* The code requires significant improvements to address security vulnerabilities (password storage) and to be suitable for a production environment. The suggested improvements must be implemented before deployment.**

FIGURE 6.12 – Capture d'écran : Code généré pour le système de gestion des tâches, non validé

## Résultats du Correction du code



The screenshot shows a code editor interface with a header bar that says "Code Corrigé" (Corrected Code) next to a wrench icon. Below the header, a light blue banner contains the text "Version optimisée et corrigée du code généré" (Optimized and corrected version of the generated code). The main area of the editor displays the following Java code:

```
```java
import java.util.Date;
import java.util.List;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.util.ArrayList;

enum Priority {
    HIGH, MEDIUM, LOW
}

enum Status {
    OPEN, IN_PROGRESS, COMPLETED
}

class Task {
    private Long id;
    private String title;
    private String description;
    private Date dueDate;
    private Priority priority;
    private Status status;
    private String assignedUser;

    public Task(String title) {
        this(title, Priority.MEDIUM, Status.OPEN);
    }

    public Task(String title, Priority priority, Status status) {
        if (title == null || title.isEmpty()) {
            throw new IllegalArgumentException("title cannot be null or empty.");
        }
        this.title = title;
        this.priority = priority;
        this.status = status;
    }

    // Getters and setters (same as before)
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }
}
```

FIGURE 6.13 – Capture d’écran : Code corrigé après validation réussie

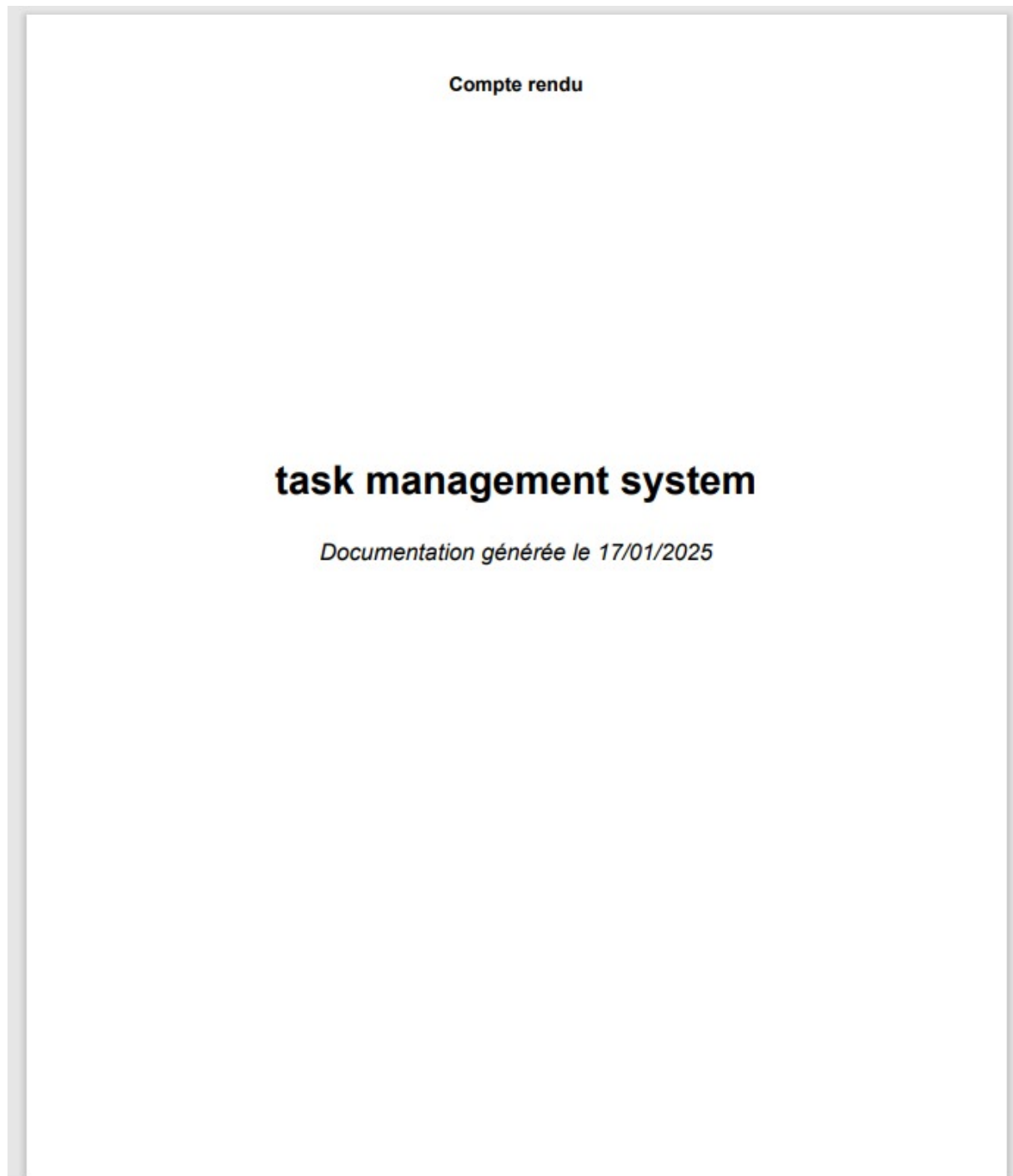


FIGURE 6.14 – Capture d’écran : première page de code générée.

6.2 Discussion des résultats et améliorations futures

6.2.1 Discussion des résultats

Le projet "Conversion d’Exigences en Logiciels Fonctionnels en Java avec LLM et Agents Intelligents" a montré des résultats convaincants en automatisant la génération de logiciels à partir des exigences en langage naturel. Le système a réussi à accomplir les tâches suivantes de manière satisfaisante :

- **Analyse des exigences** : L’agent d’analyse a correctement extrait les besoins des spécifications, prouvant ainsi une bonne capacité de compréhension du langage naturel.
- **Planification des tâches** : L’agent de planification a correctement organisé les étapes nécessaires à la réalisation du logiciel, assurant une transformation cohérente des exigences en actions.
- **Génération du code** : Le code Java généré par l’agent générateur de code correspondait aux spécifications et a été validé avec succès dans des cas simples.
- **Validation et correction du code** : L’agent de validation a permis de corriger le code généré lorsqu’il y avait des erreurs, garantissant sa conformité avec les exigences initiales.
- **Documentation générée** : La documentation a été générée automatiquement, fournissant des détails utiles sur le code, ce qui facilite la compréhension et la maintenance du logiciel.

6.2.2 Améliorations possibles et mises à jour futures

Malgré les bons résultats obtenus, plusieurs axes d’amélioration existent pour rendre le système encore plus robuste et fonctionnel :

- **Exécution du code généré** : Actuellement, le système génère du code Java qui doit être exécuté manuellement. Une amélioration importante consisterait à ajouter la possibilité d’exécuter automatiquement le code généré à partir de l’agent, directement dans l’interface de l’application. Cela permettrait de tester immédiatement la fonctionnalité du code généré, et d’afficher les résultats ou les erreurs rencontrées.
- **Amélioration de la compréhension du langage naturel** : L’intégration de modèles plus avancés de traitement du langage naturel (comme ceux basés sur GPT-4) pourrait améliorer la compréhension des exigences, en particulier pour des demandes plus complexes ou ambiguës.
- **Optimisation de la génération du code** : Bien que le code généré soit fonctionnel, il pourrait être amélioré en termes de lisibilité, de performance et de conformité aux normes de programmation. L’introduction d’un analyseur de qualité du code pour évaluer des critères comme la complexité cyclomatique ou la duplication de code serait bénéfique.
- **Personnalisation du générateur de code** : Le générateur de code pourrait être ajusté pour produire des codes spécifiques à certains frameworks ou selon des conventions de programmation particulières. Cela offrirait une meilleure flexibilité pour adapter le système à des projets réels.
- **Amélioration de la gestion des erreurs** : Bien que l’agent de validation fonctionne correctement pour détecter des erreurs simples, une gestion plus fine des erreurs pourrait être introduite, notamment pour les erreurs de logique ou d’architecture. Une meilleure capacité à suggérer des corrections contextuelles améliorerait l’expérience utilisateur.
- **Amélioration de la documentation** : La documentation générée pourrait être enrichie en ajoutant des exemples d’utilisation plus détaillés, ainsi qu’une gestion plus avancée des erreurs et des commentaires dans le code généré. De plus, une documentation interactive générée en temps réel pourrait rendre l’outil encore plus utile.
- **Extension à d’autres langages de programmation** : Bien que le projet soit centré sur Java, il serait pertinent d’étendre le système à d’autres langages comme Python, C++, ou JavaScript, ce qui élargirait son champ d’application.
- **Interface utilisateur plus interactive** : L’interface web pourrait être rendue

plus interactive, avec des visualisations des différentes étapes de la génération du code et de la validation. Une interface qui permettrait également d'interagir avec les agents, comme la modification des exigences ou l'exécution directe du code, serait un atout majeur.

- **Automatisation des tests unitaires et d'intégration** : L'intégration de tests automatisés pour valider le code généré dans des situations réelles permettrait de s'assurer que le système fonctionne de manière fiable dans tous les cas d'utilisation.

Chapitre 7

Conclusion générale

Le projet intitulé **Conversion d'Exigences en Logiciels Fonctionnels en Java avec LLM et Agents Intelligents** a permis de démontrer le potentiel des technologies d'intelligence artificielle pour automatiser la création de logiciels à partir des exigences en langage naturel. Grâce à l'intégration des modèles de langage de grande taille (LLM) et de systèmes multi-agents, notre application est capable de convertir des spécifications simples en code fonctionnel, tout en validant et en générant de la documentation.

Les résultats obtenus ont montré que les différents agents fonctionnaient de manière cohérente, depuis l'analyse des exigences jusqu'à la validation et la génération du code. L'application a permis de générer des solutions logicielles en Java, offrant ainsi un outil puissant pour la création rapide de logiciels à partir de descriptions fonctionnelles en langage naturel.

Cependant, plusieurs améliorations peuvent être apportées afin d'augmenter la robustesse et l'efficacité du système. Parmi les pistes d'amélioration, nous envisageons l'ajout de la possibilité d'exécuter automatiquement le code généré, l'extension à d'autres langages de programmation, et l'amélioration de la gestion des erreurs et de la personnalisation du générateur de code. Une interface plus interactive et la mise en place de tests automatisés sont également des éléments à explorer pour rendre l'application plus user-friendly et adaptable à différents types de projets.

Dans l'ensemble, ce projet a démontré le potentiel des agents intelligents pour faciliter le développement logiciel et offrir une approche innovante dans la gestion des exigences et la création automatique de code. Nous espérons que les améliorations futures permettront de rendre cet outil encore plus puissant et polyvalent, avec une application possible dans des contextes industriels et professionnels.

Chapitre 8

Références

- 1 <https://ai.google.dev/Google AI>. "AI Platform Overview."
- 2 <https://docs.crewai.com/introductionCrew AI Documentation>. "Introduction to Crew AI."
- 3 <https://www.youtube.com/watch?v=KsMTF6bjSzot=2920s>Vidéo explicative sur le fonctionnement de Crew AI.
- 4 <https://github.com/krishnaik06/CrewAI-Projects/tree/main/crewgooglegeminiInspiration> pour la structure de code. CrewAI Projects sur GitHub.